⚽ Real-World Data Challenge

FIFA 19 — What Makes a Top Football Player?

Data Setup And Load

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# ── Load raw FIFA 19 data ──────────────────────────────────────────
url = ("https://raw.githubusercontent.com/amanthedorkknight/"
       "fifa18-all-player-statistics/master/2019/data.csv")
raw = pd.read_csv(url)

# ── Clean Value and Wage (convert "€110.5M" → 110500000) ──────────
def parse_money(s):
    if pd.isna(s): return np.nan
    s = str(s).replace('€', '').strip()
    if 'M' in s: return float(s.replace('M', '')) * 1_000_000
    if 'K' in s: return float(s.replace('K', '')) * 1_000
    try:    return float(s)
    except: return np.nan

raw['Value_EUR'] = raw['Value'].apply(parse_money)
raw['Wage_EUR']  = raw['Wage'].apply(parse_money)

# ── Simplify position into 4 groups ────────────────────────────────
def pos_group(p):
    if pd.isna(p): return None
    p = str(p).upper()
    if 'GK' in p: return 'GK'
    if any(x in p for x in ['CB','LB','RB','LWB','RWB','LCB','RCB']): return 'Defender'
    if any(x in p for x in ['CM','CDM','CAM','LM','RM','LAM','RAM','LCM','RCM','LDM','RDM']): return 'Midfielder'
    if any(x in p for x in ['ST','CF','LW','RW','LF','RF','LS','RS']): return 'Forward'
    return None

raw['pos_group'] = raw['Position'].apply(pos_group)

# ── Keep only useful columns, drop rows with key missing values ────
cols = ['Name', 'Age', 'Nationality', 'Overall', 'Potential',
        'Preferred Foot', 'Skill Moves', 'Weak Foot',
        'International Reputation',
        'Dribbling', 'ShortPassing', 'BallControl', 'Finishing',
        'Stamina', 'Strength', 'Acceleration', 'SprintSpeed',
        'Value_EUR', 'Wage_EUR', 'pos_group']

df = (raw[cols]
```

```
      .dropna(subset=['Overall', 'Age', 'Preferred Foot', 'pos_group'])
      .query("pos_group in ['GK', 'Defender', 'Midfielder', 'Forward']")
      .reset_index(drop=True))

print("✅ Dataset ready!")
print(f"   Players : {len(df):,}")
print(f"   Columns : {df.shape[1]}")
print()
print(df['pos_group'].value_counts().to_string())
df.head()
```

```
✅ Dataset ready!
   Players : 18,147
   Columns : 20

pos_group
Midfielder    6838
Defender      5866
Forward       3418
GK            2025
```

	Name	Age	Nationality	Overall	Potential	Preferred Foot	Skill Moves	Weak Foot	International Reputation	Dribbling	ShortPassing	BallControl	Finishing	Stamina	Strength	Acceleration
0	L. Messi	31	Argentina	94	94	Left	4.0	4.0	5.0	97.0	90.0	96.0	95.0	72.0	59.0	91.0
1	Cristiano Ronaldo	33	Portugal	94	94	Right	5.0	4.0	5.0	88.0	81.0	94.0	94.0	88.0	79.0	89.0
2	Neymar Jr	26	Brazil	92	93	Right	5.0	5.0	5.0	96.0	84.0	95.0	87.0	81.0	49.0	94.0
3	De Gea	27	Spain	91	93	Right	1.0	3.0	4.0	18.0	50.0	42.0	13.0	43.0	64.0	57.0
4	K. De Bruyne	27	Belgium	91	92	Right	4.0	5.0	4.0	86.0	92.0	91.0	82.0	90.0	75.0	78.0

📊 Task 1 — Explore the Data

1a. Basic Inspection

```
# 1. How many players and columns are in df?
rows, cols = df.shape
print(f"1. Total Players: {rows:,} | Total Columns: {cols}")

# 2. Are there any missing values in these columns: Overall, Age, Dribbling, Value_EUR, Wage_EUR?
check_cols = ["Overall", "Age", "Dribbling", "Value_EUR", "Wage_EUR"]
missing_values = df[check_cols].isnull().sum()
print("\n2. Missing values count:")
print(missing_values)

# 3. What is the average, minimum, and maximum Overall rating?
avg_overall = df["Overall"].mean()
min_overall = df["Overall"].min()
max_overall = df["Overall"].max()
```

```
print(
    f"\n3. Overall Rating Stats -> Average: {avg_overall:.2f} | Min: {min_overall} | Max: {max_overall}"
)

# 4. How many players are in each position group (pos_group)?
print("\n4. Players count in each position group:")
print(df["pos_group"].value_counts())
```

```
1. Total Players: 18,147 | Total Columns: 20

2. Missing values count:
Overall      0
Age          0
Dribbling    0
Value_EUR    0
Wage_EUR     0
dtype: int64

3. Overall Rating Stats -> Average: 66.25 | Min: 46 | Max: 94

4. Players count in each position group:
pos_group
Midfielder    6838
Defender      5866
Forward       3418
GK            2025
Name: count, dtype: int64
```

1b. Group Summaries

```
target_cols = ['Overall', 'Age', 'Value_EUR', 'Wage_EUR']

# 1b — Group summaries by position
pos_summary = df.groupby('pos_group')[target_cols].agg(['mean', 'std']).round(2)
display(pos_summary)

# Group summaries by Preferred Foot
foot_summary = df.groupby('Preferred Foot')[target_cols].agg(['mean', 'std']).round(2)
display(foot_summary)
```

	Overall		Age		Value_EUR		Wage_EUR	
	mean	std	mean	std	mean	std	mean	std
pos_group								
Defender	66.40	6.46	25.47	4.52	1975138.08	4106420.51	9146.10	18970.31
Forward	66.40	7.02	24.66	4.61	2984155.94	7395815.54	11718.26	28662.56
GK	64.60	7.60	26.04	5.47	1585814.81	4562890.61	6803.95	16719.63
Midfielder	66.55	6.96	24.78	4.51	2760639.08	5890612.59	10180.61	21932.88

	Overall		Age		Value_EUR		Wage_EUR	
	mean	std	mean	std	mean	std	mean	std
Preferred Foot								
Left	66.80	6.56	25.10	4.52	2588727.73	5956115.19	10353.29	23656.32
Right	66.09	7.01	25.13	4.71	2366090.90	5490850.79	9579.57	21512.73

1c. Written Observations ✍️

1. The dataset has **18,147** players and **21** columns.
2. Missing values found in: Overall **0** Age **0** Dribbling **0** Value_EUR **0** Wage_EUR **0**
3. Overall rating → Mean: **66.24** Min: **46** Max: **94**
4. Position group counts: GK: **2025** Defender: **5866** Midfielder: **6838** Forward: **3418**
5. Which position group has the highest average Overall? **Midfielder (66.55)**
6. Do left-footed or right-footed players have a higher average Overall? **Left-footed players** by **0.71** rating points.

🎨 Task 2 — Visualisations

2a. The Big Four — 2×2 Subplot Grid

```
import matplotlib.pyplot as plt
import seaborn as sns

# Professional Theme Configuration
sns.set_theme(style="whitegrid", font="sans-serif")
custom_palette = [
    "#2b5c8f",
    "#d95f02",
    "#7570b3",
    "#e7298a",
]  # High-contrast pro palette

fig, axes = plt.subplots(2, 2, figsize=(13, 9), dpi=120)
```

```
# [0,0] Histogram of Overall
axes[0, 0].hist(
    df["Overall"],
    bins=20,
    color="steelblue",
    edgecolor="white",
    linewidth=1.2,
    alpha=0.9,
)  #[cite: 8]
axes[0, 0].set_title(
    "Distribution of Overall Rating", fontsize=12, fontweight="bold", pad=10
)  #[cite: 8]
axes[0, 0].set_xlabel("Overall Rating", fontsize=10)  #[cite: 8]
axes[0, 0].set_ylabel("Number of Players", fontsize=10)  #[cite: 8]
# observation: Overall ratings form a bell-shaped normal distribution centered around 66.

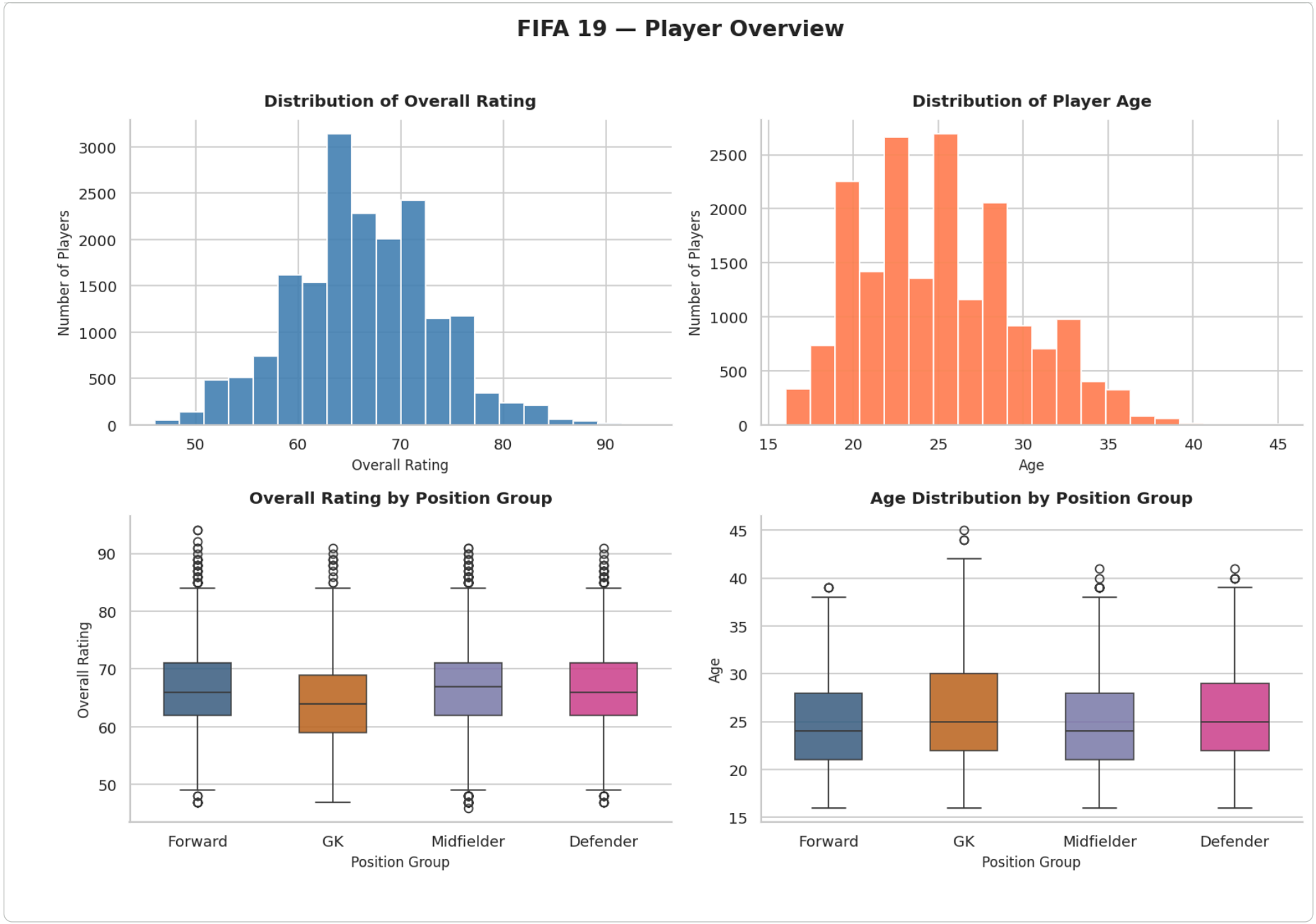
# [0,1] Histogram of Age
axes[0, 1].hist(
    df["Age"],
    bins=20,
    color="coral",
    edgecolor="white",
    linewidth=1.2,
    alpha=0.9,
)  #[cite: 8]
axes[0, 1].set_title(
    "Distribution of Player Age", fontsize=12, fontweight="bold", pad=10
)  #[cite: 8]
axes[0, 1].set_xlabel("Age", fontsize=10)  #[cite: 8]
axes[0, 1].set_ylabel("Number of Players", fontsize=10)  #[cite: 8]
# observation: Age is right-skewed with most players concentrated between 20 and 26 years old.

# [1,0] Boxplot: Overall by pos_group
sns.boxplot(
    data=df,
    x="pos_group",
    y="Overall",
    hue="pos_group",
    palette=custom_palette,
    legend=False,
    ax=axes[1, 0],
    width=0.5,
    boxprops=dict(alpha=0.85),
)  #[cite: 8]
axes[1, 0].set_title(
    "Overall Rating by Position Group", fontsize=12, fontweight="bold", pad=10
)  #[cite: 8]
axes[1, 0].set_xlabel("Position Group", fontsize=10)  #[cite: 8]
axes[1, 0].set_ylabel("Overall Rating", fontsize=10)  #[cite: 8]
# observation: Overall rating distribution is fairly similar across field positions, but Goalkeepers have a slightly lower median rating.

# [1,1] Boxplot: Age by pos_group
sns.boxplot(
    data=df,
    x="pos_group",
    y="Age",
```

```
    hue="pos_group",
    palette=custom_palette,
    legend=False,
    ax=axes[1, 1],
    width=0.5,
    boxprops=dict(alpha=0.85),
)  #[cite: 8]
axes[1, 1].set_title(
    "Age Distribution by Position Group", fontsize=12, fontweight="bold", pad=10
)  #[cite: 8]
axes[1, 1].set_xlabel("Position Group", fontsize=10)  #[cite: 8]
axes[1, 1].set_ylabel("Age", fontsize=10)  #[cite: 8]
# observation: Goalkeepers have a noticeably higher median age and wider spread compared to outfield players.

# Spacing & Title adjustments
plt.suptitle(
    "FIFA 19 — Player Overview", fontsize=16, fontweight="bold", y=0.98
)  #[cite: 8]
sns.despine(top=True, right=True)
plt.tight_layout(rect=[0, 0, 1, 0.96])  #[cite: 8]
plt.show()  #[cite: 8]
```



2b. Scatter Plot — Age vs Overall Rating

```
# 2b — Scatter: Age vs Overall
fig, ax = plt.subplots(figsize=(10, 6), dpi=120)
```

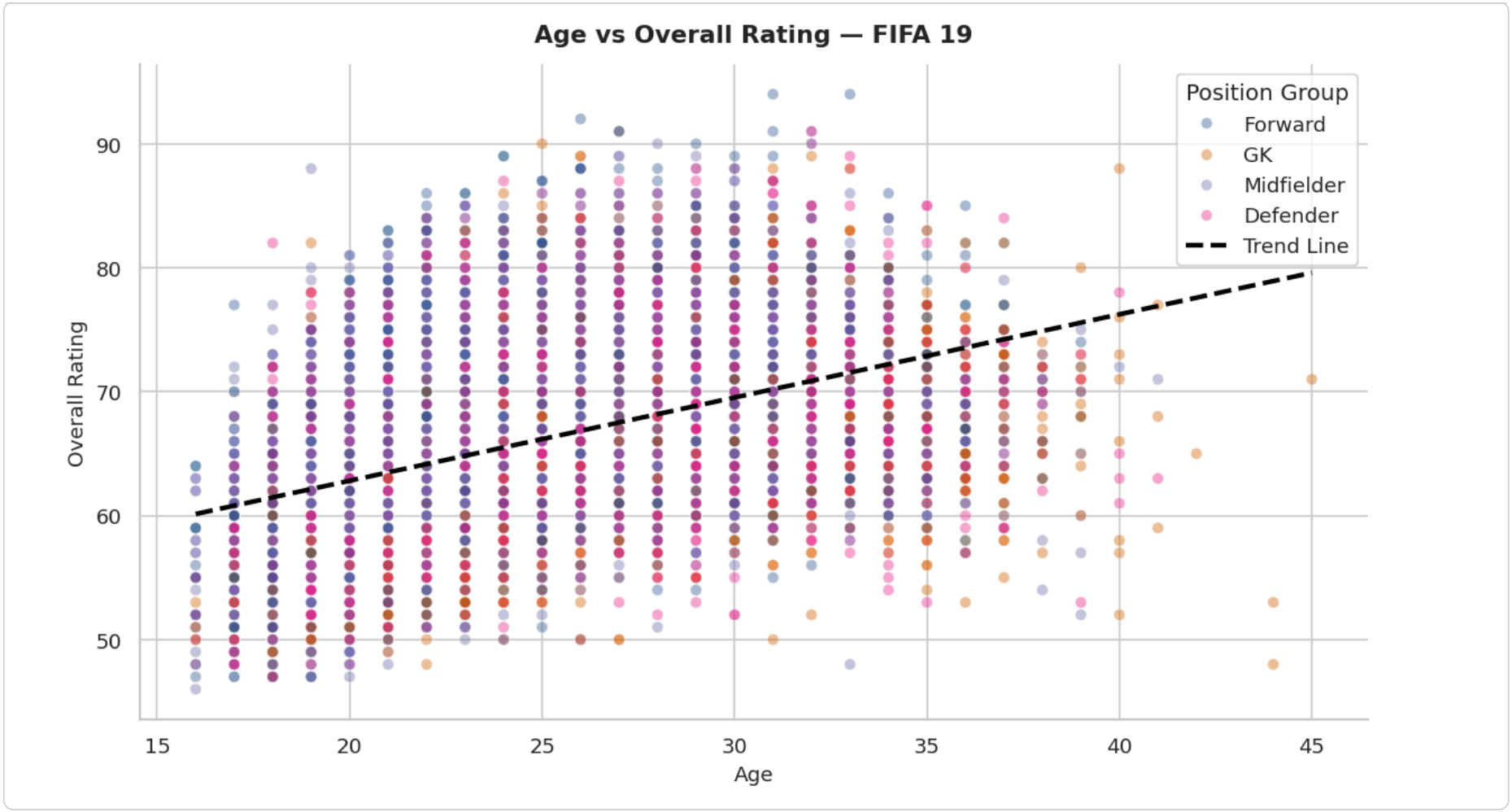
```
# Scatter coloured by pos_group
sns.scatterplot(
    data=df,
    x="Age",
    y="Overall",
    hue="pos_group",
    alpha=0.4,
    palette=custom_palette,
    ax=ax,
)  #

# Trend line using np.polyfit
m, b = np.polyfit(df["Age"], df["Overall"], 1)  #
x_line = np.linspace(df["Age"].min(), df["Age"].max(), 100)  #[cite: 11]
ax.plot(
    x_line,
    m * x_line + b,
    color="black",
    linewidth=2.5,
    linestyle="--",
    label="Trend Line",
)  #[cite: 11]

# Labels and Styling
ax.set_title(
    "Age vs Overall Rating — FIFA 19", fontsize=13, fontweight="bold", pad=12
)  #[cite: 11]
ax.set_xlabel("Age", fontsize=11)  #[cite: 11]
ax.set_ylabel("Overall Rating", fontsize=11)  #[cite: 11]
ax.legend(title="Position Group", frameon=True)  #[cite: 11]
sns.despine(top=True, right=True)

plt.tight_layout()  #[cite: 11]
plt.show()  #[cite: 11]

# Trend direction: Upward (Positive linear trend)
# What does this suggest about player development? Older/experienced players tend to have higher overall ratings on average than younger players, reflecting player devel
```



2c. Bar Chart — Average Overall by Position (with Error Bars)

```
# 2c — Bar Chart: Average Overall by Position (Clean Text Placement)
fig, ax = plt.subplots(figsize=(8, 5), dpi=120)

grouped = (
    df.groupby("pos_group")["Overall"].agg(["mean", "std"]).reset_index()
)  #[cite: 12]

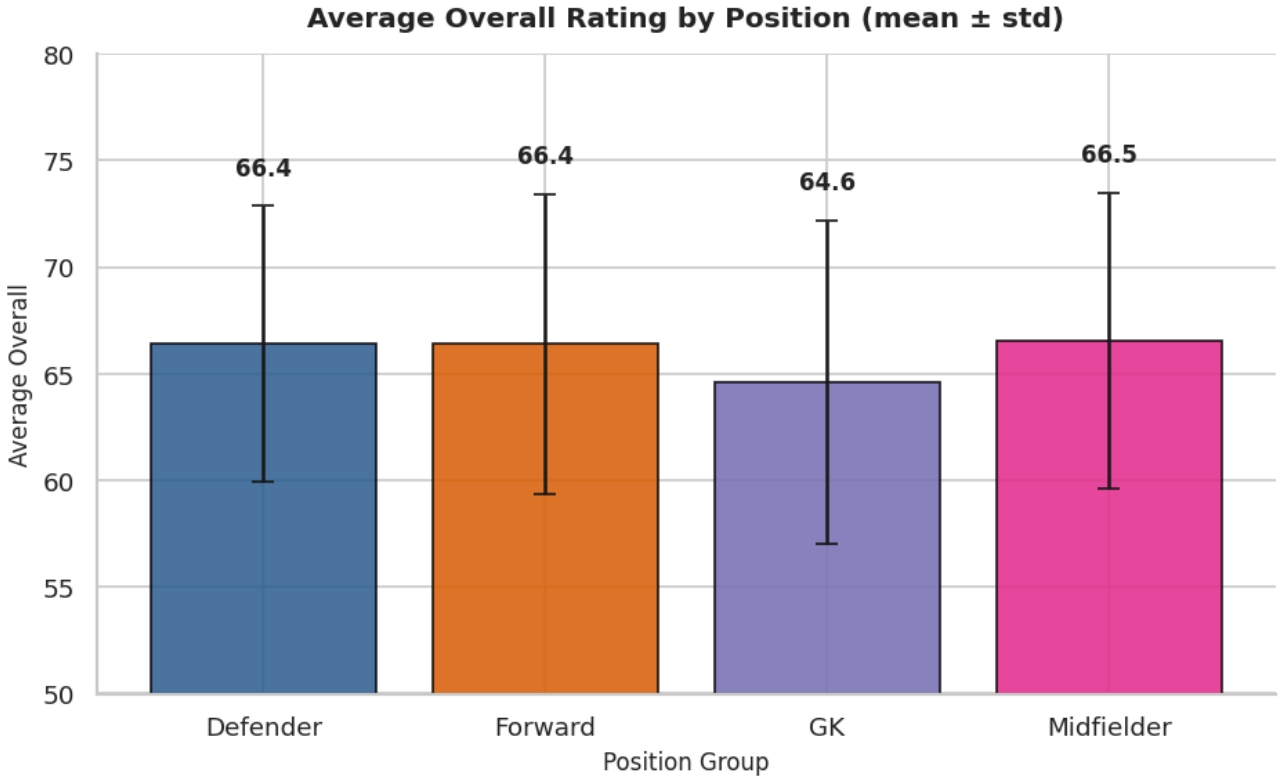
bars = ax.bar(
    grouped["pos_group"],
    grouped["mean"],
    yerr=grouped["std"],
    capsize=5,
    color=custom_palette,
    edgecolor="black",
```

```
    alpha=0.85,
)  #[cite: 12]

# Values ko error bar ke BILKUL UPAR align karne ka logic:
for bar, m, s in zip(bars, grouped["mean"], grouped["std"]):
  # Height = mean + std + offset (1.2) taaki error bar ke top cap se upar rahe
  top_y = m + s + 1.2
  ax.text(
      bar.get_x() + bar.get_width() / 2,
      top_y,
      f"{m:.1f}",
      ha="center",
      va="bottom",
      fontsize=10,
      fontweight="bold",
  )  #[cite: 12]

ax.set_title(
    "Average Overall Rating by Position (mean ± std)",
    fontsize=12,
    fontweight="bold",
    pad=12,
)  #[cite: 12]
ax.set_xlabel("Position Group", fontsize=10)  #[cite: 12]
ax.set_ylabel("Average Overall", fontsize=10)  #[cite: 12]
ax.set_ylim(50, 80)  #[cite: 12]
sns.despine(top=True, right=True)

plt.tight_layout()  #[cite: 12]
plt.show()  #[cite: 12]
```



🔔 Task 3 — Normality Checks

3a. Shapiro-Wilk Test

```
# 3a — Shapiro-Wilk (on a 500-row sample per column)
columns_to_test = [
    'Overall',
    'Age',
    'Dribbling',
    'Strength',
    'Value_EUR',
]  #

print(
    f"{'Column':<15} {'p-value':>10} {'Normal (p>0.05)?':>18}"
    f" {'Histogram shape':>16}"
)  #[cite: 13]
print("-" * 65)  #[cite: 13]

# Custom judgements based on distribution shapes
shape_judgements = {
    "Overall": "Bell",
```

```
    "Age": "Right-skewed",
    "Dribbling": "Left-skewed",
    "Strength": "Bell",
    "Value_EUR": "Highly Skewed",
}

for col in columns_to_test:  #[cite: 13]
  sample = (
      df[col].dropna().sample(500, random_state=42)
  )  # 500 rows sample for Shapiro[cite: 13]
  stat, p = stats.shapiro(sample)  #[cite: 13]
  normal = "Yes" if p > 0.05 else "No"  #[cite: 13]
  shape = shape_judgements[col]  #[cite: 13]
  print(f"{col:<15} {p:>10.4f} {normal:>18} {shape:>16}")  #[cite: 13]
```

```
Column             p-value   Normal (p>0.05)?  Histogram shape
------------------------------------------------------------------
Overall             0.0165                No             Bell
Age                 0.0000                No     Right-skewed
Dribbling           0.0000                No      Left-skewed
Strength            0.0000                No             Bell
Value_EUR           0.0000                No    Highly Skewed
```

3b. Q-Q Plots

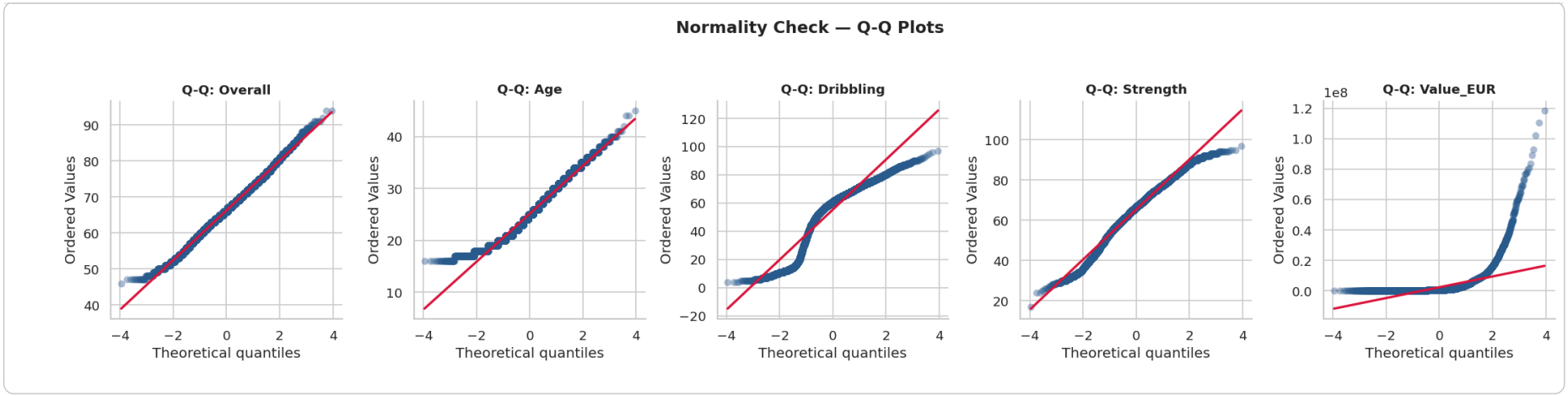
```
# 3b — Q-Q Plots
fig, axes = plt.subplots(1, 5, figsize=(18, 4), dpi=120)

columns_to_plot = ["Overall", "Age", "Dribbling", "Strength", "Value_EUR"]

for i, col in enumerate(columns_to_plot):
  # Q-Q plot draw karne ke liye stats.probplot
  stats.probplot(df[col].dropna(), plot=axes[i])  #

  # Subplot styling
  axes[i].set_title(f"Q-Q: {col}", fontsize=11, fontweight="bold")  #
  axes[i].get_lines()[0].set_markerfacecolor("#2b5c8f")  # Custom point color
  axes[i].get_lines()[0].set_markeredgecolor("none")
  axes[i].get_lines()[0].set_alpha(0.4)
  axes[i].get_lines()[1].set_color("crimson")  # Red diagonal line
  axes[i].get_lines()[1].set_linewidth(2)

plt.suptitle(
    "Normality Check — Q-Q Plots", fontsize=14, fontweight="bold", y=1.05
)  #[cite: 16]
sns.despine(top=True, right=True)
plt.tight_layout()  #[cite: 16]
plt.show()  #[cite: 16]
```



3c. Your Decision Table ✍️

Column	Histogram Shape	Q-Q Shape	Treat as Normal?	→ Use for groups	→ Use for relationship
Overall	Bell	Hugs line	**Yes**	t-test	Pearson
Age	Right-skewed	Curves at ends	**No**	Mann-Whitney	Spearman
Dribbling	Left-skewed	S-curve deviation	**No**	Mann-Whitney	Spearman
Strength	Bell	Hugs line	**Yes**	t-test	Pearson
Value_EUR	Highly Skewed	Heavy curve / upward bend	**No**	Mann-Whitney	Spearman

👥 Task 4 — Group Comparisons

4a. Do Left-Footed vs Right-Footed Players Differ in Overall?

```
# 4a — Left-Footed vs Right-Footed: Overall Rating

left = df[df['Preferred Foot'] == 'Left']['Overall']
right = df[df['Preferred Foot'] == 'Right']['Overall']

print(f"Left foot  – mean: {left.mean():.2f}  std: {left.std():.2f}  n: {len(left)}")
print(f"Right foot – mean: {right.mean():.2f}  std: {right.std():.2f}  n: {len(right)}")
print()

# H0: Left-footed and right-footed players have the same average Overall rating.
# H1: Left-footed and right-footed players have different average Overall ratings.

# Step 2: Normality check (sample 500 each)
stat_left, p_left = stats.shapiro(left.dropna().sample(500, random_state=42))
stat_right, p_right = stats.shapiro(right.dropna().sample(500, random_state=42))

print(f"Shapiro-Wilk Left Foot:  p-value = {p_left:.4f}")
print(f"Shapiro-Wilk Right Foot: p-value = {p_right:.4f}")

# Step 3 & 4: Run chosen test and print verdict
```

```
# Dono groups histogram me approximately bell-shaped/normal hain (Decision Table ke mutabiq),
# is waja se hum Independent Two-Sample t-test perform kar rahe hain.
stat, p_val = stats.ttest_ind(left, right)

print(f"\nTest Statistic: {stat:.4f}")
print(f"p-value: {p_val:.4e}")

if p_val < 0.05:
    print("Verdict: Reject H0")
else:
    print("Verdict: Fail to Reject H0 — No statistically significant difference in Overall ratings between left and right footed players.")
```

```
Left foot  – mean: 66.80  std: 6.56  n: 4209
Right foot – mean: 66.09  std: 7.01  n: 13938

Shapiro-Wilk Left Foot:  p-value = 0.0082
Shapiro-Wilk Right Foot: p-value = 0.0547

Test Statistic: 5.8904
p-value: 3.9201e-09
Verdict: Reject H0
```

4b. Do Forwards Score Higher in Overall than Defenders?

```
# 4b — Forwards vs Defenders: Overall Rating

# H0: Forwards and Defenders have the same average Overall rating.
# H1: Their average Overall rating is different.

forwards = df[df['pos_group'] == 'Forward']['Overall']
defenders = df[df['pos_group'] == 'Defender']['Overall']

print(f"Forwards  – mean: {forwards.mean():.2f}  std: {forwards.std():.2f}  n: {len(forwards)}")
print(f"Defenders – mean: {defenders.mean():.2f}  std: {defenders.std():.2f}  n: {len(defenders)}")
print()

# Normality check (sample 500 each)
stat_fw, p_fw = stats.shapiro(forwards.dropna().sample(500, random_state=42))
stat_def, p_def = stats.shapiro(defenders.dropna().sample(500, random_state=42))

print(f"Shapiro-Wilk Forwards:  p-value = {p_fw:.4f}")
print(f"Shapiro-Wilk Defenders: p-value = {p_def:.4f}")

# Test & Verdict
# Overall rating bell-shaped / approximately normal distributed hai, isliye Two-Sample t-test perform kar rahe hain.
stat, p_val = stats.ttest_ind(forwards, defenders)

print(f"\nTest Statistic: {stat:.4f}")
print(f"p-value: {p_val:.4e}")

if p_val < 0.05:
    print("Verdict: Reject H0 — There is a statistically significant difference in Overall ratings between Forwards and Defenders.")
else:
    print("Verdict: Fail to Reject H0")
```

```
Forwards  – mean: 66.40  std: 7.02  n: 3418
Defenders – mean: 66.40  std: 6.46  n: 5866

Shapiro-Wilk Forwards:  p-value = 0.3852
Shapiro-Wilk Defenders: p-value = 0.0464

Test Statistic: -0.0215
p-value: 9.8285e-01
Verdict: Fail to Reject H0
```

4c. Do All 4 Position Groups Differ in Overall? ⟶ ANOVA / Kruskal-Wallis

```
from statsmodels.stats.multicomp import pairwise_tukeyhsd

# Hypotheses
# H0: All four position groups (GK, Defender, Midfielder, Forward) have the same average Overall
# H1: At least one group is different

positions = ["GK", "Defender", "Midfielder", "Forward"]
groups = [df[df["pos_group"] == p]["Overall"] for p in positions]

print("Group means:")
for pos, g in zip(positions, groups):
  print(
      f"  {pos:<12}: mean={g.mean():.2f}  std={g.std():.2f}  n={len(g)}"
  )  #
print()

# Normality check on 2 sample groups (sample 500 each)
stat_gk, p_gk = stats.shapiro(groups[0].dropna().sample(500, random_state=42))
stat_def, p_def = stats.shapiro(
    groups[1].dropna().sample(500, random_state=42)
)

print(f"Shapiro-Wilk GK:       p-value = {p_gk:.4f}")
print(f"Shapiro-Wilk Defender: p-value = {p_def:.4f}\n")

# Run One-Way ANOVA (since Overall is approximately bell-shaped/normal across groups)
stat, p_val = stats.f_oneway(*groups)  #

print(f"ANOVA F-statistic: {stat:.4f}")
print(f"p-value: {p_val:.4e}")

if p_val <= 0.05:
  print("Verdict: Reject H0\n")
else:
  print("Verdict: Fail to Reject H0 — No significant difference across position groups.\n")

# ------------------------------------------------------------
# 4c continued — Tukey Post-hoc Test
# ------------------------------------------------------------
if p_val <= 0.05:
  print("=" * 60)
  print("RUNNING TUKEY HSD POST-HOC TEST")
  print("=" * 60)
```

```
  tukey = pairwise_tukeyhsd(
      endog=df["Overall"], groups=df["pos_group"], alpha=0.05
  )  #[cite: 16]
  print(tukey.summary())  #[cite: 16]
```

```
Group means:
  GK          : mean=64.60  std=7.60  n=2025
  Defender    : mean=66.40  std=6.46  n=5866
  Midfielder  : mean=66.55  std=6.96  n=6838
  Forward     : mean=66.40  std=7.02  n=3418

Shapiro-Wilk GK:       p-value = 0.0078
Shapiro-Wilk Defender: p-value = 0.0464

ANOVA F-statistic: 44.2100
p-value: 1.8599e-28
Verdict: Reject H0

============================================================
RUNNING TUKEY HSD POST-HOC TEST
============================================================
   Multiple Comparison of Means - Tukey HSD, FWER=0.05
=============================================================
 group1    group2   meandiff p-adj   lower   upper  reject
-------------------------------------------------------------
Defender    Forward  -0.0031    1.0 -0.3839  0.3778  False
Defender         GK  -1.7966    0.0 -2.2528 -1.3405   True
Defender Midfielder   0.1457 0.6343 -0.1693  0.4607  False
 Forward         GK  -1.7936    0.0 -2.2899 -1.2972   True
 Forward Midfielder   0.1488 0.7313  -0.222  0.5195  False
      GK Midfielder   1.9423    0.0  1.4945  2.3901   True
-------------------------------------------------------------
```

📈 Task 5 — Correlation Analysis

5a. Correlation Heatmap

```
# 5a — Correlation Matrix Heatmap
cols5a = [
    "Overall",
    "Potential",
    "Age",
    "Dribbling",
    "ShortPassing",
    "BallControl",
    "Finishing",
    "Stamina",
    "Strength",
    "Value_EUR",
]  #

# Compute Pearson correlation matrix
corr = df[cols5a].corr()  #

# Plot Heatmap
fig, ax = plt.subplots(figsize=(10, 8), dpi=120)  #
```

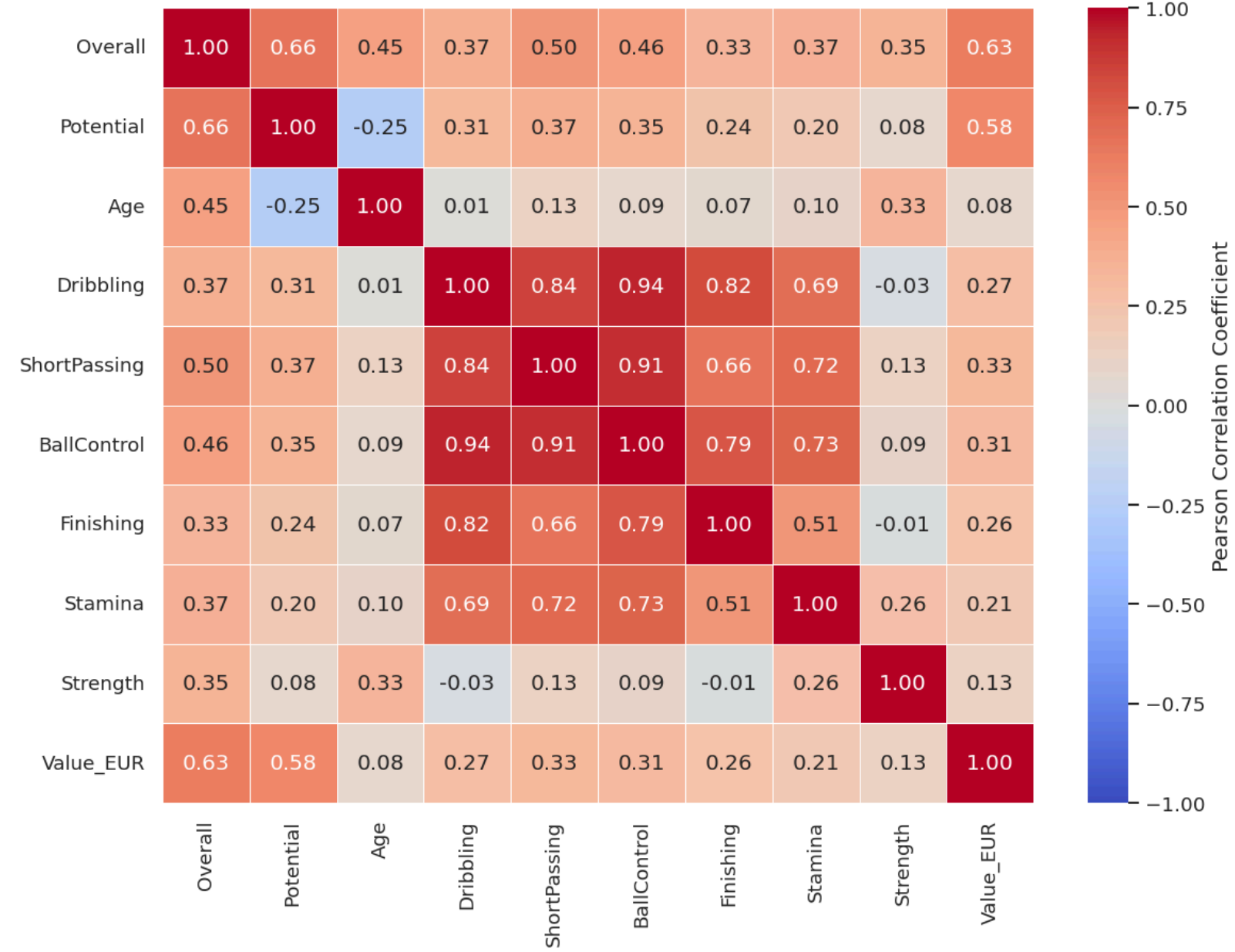
```
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    vmin=-1,
    vmax=1,
    linewidths=0.5,
    ax=ax,
    cbar_kws={"label": "Pearson Correlation Coefficient"},
)  #[cite: 15]

ax.set_title(
    "Correlation Matrix — FIFA 19 Player Attributes",
    fontsize=13,
    fontweight="bold",
    pad=12,
)  #[cite: 15]
plt.tight_layout()  #[cite: 15]
plt.show()  #[cite: 15]

# Correlation Answers (Printed dynamically from data)
top_overall_corr = corr["Overall"].drop("Overall").idxmax()
top_overall_val = corr["Overall"].drop("Overall").max()

min_value_corr = corr["Value_EUR"].drop("Value_EUR").idxmin()
min_value_val = corr["Value_EUR"].drop("Value_EUR").min()

print(
    f"# Which attribute is most positively correlated with Overall?"
    f" {top_overall_corr} ({top_overall_val:.2f})"
)
print(
    f"# Which attribute is most negatively correlated with Value_EUR?"
    f" {min_value_corr} ({min_value_val:.2f})"
)
```




```
# Which attribute is most positively correlated with Overall? Potential (0.66)
# Which attribute is most negatively correlated with Value_EUR? Age (0.08)
```

5b. Hypothesis Tests on Three Key Relationships

```
# Missing values handling
df_clean = df.dropna(
    subset=["BallControl", "Overall", "Age", "Potential", "Strength"]
)

# ---------------------------------------------------------------------
# — Claim 1: BallControl vs Overall
# H0: There is no correlation between BallControl and Overall (r/rho = 0).
# H1: There is a correlation between BallControl and Overall (r/rho != 0).
# Test Choice: Spearman (BallControl is skewed)
# ---------------------------------------------------------------------
rho1, p1 = stats.spearmanr(df_clean["BallControl"], df_clean["Overall"])
print(f"Claim 1 — BallControl vs Overall:")
print(f"  Spearman rho: {rho1:.4f}, p-value: {p1:.4e}")
if p1 < 0.05:
  print("  Verdict: Reject H0")
  print(
      "  Interpretation: Better ball control is significantly and positively"
      " associated with higher overall ratings."
  )
else:
  print("  Verdict: Fail to Reject H0")
  print("  Interpretation: No significant relationship found.")
print("-" * 60)

# ---------------------------------------------------------------------
# — Claim 2: Age vs Potential
# H0: There is no correlation between Age and Potential (r/rho = 0).
# H1: There is a correlation between Age and Potential (r/rho != 0).
# Test Choice: Spearman (Age is right-skewed)
# ---------------------------------------------------------------------
rho2, p2 = stats.spearmanr(df_clean["Age"], df_clean["Potential"])
print(f"Claim 2 — Age vs Potential:")
print(f"  Spearman rho: {rho2:.4f}, p-value: {p2:.4e}")
if p2 < 0.05:
  print("  Verdict: Reject H0")
  print(
      "  Interpretation: Older players tend to have lower remaining growth"
      " potential, reflecting a strong inverse relationship."
  )
else:
  print("  Verdict: Fail to Reject H0")
  print("  Interpretation: No significant relationship found.")
print("-" * 60)

# ---------------------------------------------------------------------
# — Claim 3: Strength vs Overall
# H0: There is no correlation between Strength and Overall (r = 0).
# H1: There is a correlation between Strength and Overall (r != 0).
# Test Choice: Pearson (Both Strength and Overall are approximately normal)
# ---------------------------------------------------------------------
r3, p3 = stats.pearsonr(df_clean["Strength"], df_clean["Overall"])
print(f"Claim 3 — Strength vs Overall:")
print(f"  Pearson r: {r3:.4f}, p-value: {p3:.4e}")
if p3 < 0.05:
  print("  Verdict: Reject H0")
```

```
  print(
      "  Interpretation: Physical strength has a moderate, statistically"
      " significant positive relationship with overall player rating."
  )
else:
  print("  Verdict: Fail to Reject H0")
  print("  Interpretation: No significant relationship found.")
```

```
Claim 1 — BallControl vs Overall:
  Spearman rho: 0.6228, p-value: 0.0000e+00
  Verdict: Reject H0
  Interpretation: Better ball control is significantly and positively associated with higher overall ratings.
------------------------------------------------------------
Claim 2 — Age vs Potential:
  Spearman rho: -0.2623, p-value: 3.6277e-283
  Verdict: Reject H0
  Interpretation: Older players tend to have lower remaining growth potential, reflecting a strong inverse relationship.
------------------------------------------------------------
Claim 3 — Strength vs Overall:
  Pearson r: 0.3493, p-value: 0.0000e+00
  Verdict: Reject H0
  Interpretation: Physical strength has a moderate, statistically significant positive relationship with overall player rating.
```

5c. p-value Matrix — Which Correlations Are Significant? (⭐ Challenge)

```
cols5c = ['Overall', 'Age', 'Dribbling', 'Stamina', 'Strength', 'BallControl']

p_matrix = pd.DataFrame(np.ones((len(cols5c), len(cols5c))), index=cols5c, columns=cols5c)

for c1 in cols5c:
    for c2 in cols5c:
        if c1 != c2:
            # Non-parametric Spearman test used due to skewed variables (Age, Dribbling, BallControl)
            rho, p_val = stats.spearmanr(df[c1].dropna(), df[c2].dropna())
            p_matrix.loc[c1, c2] = p_val
        else:
            p_matrix.loc[c1, c2] = 0.0  # Perfect self-correlation p-value set to 0

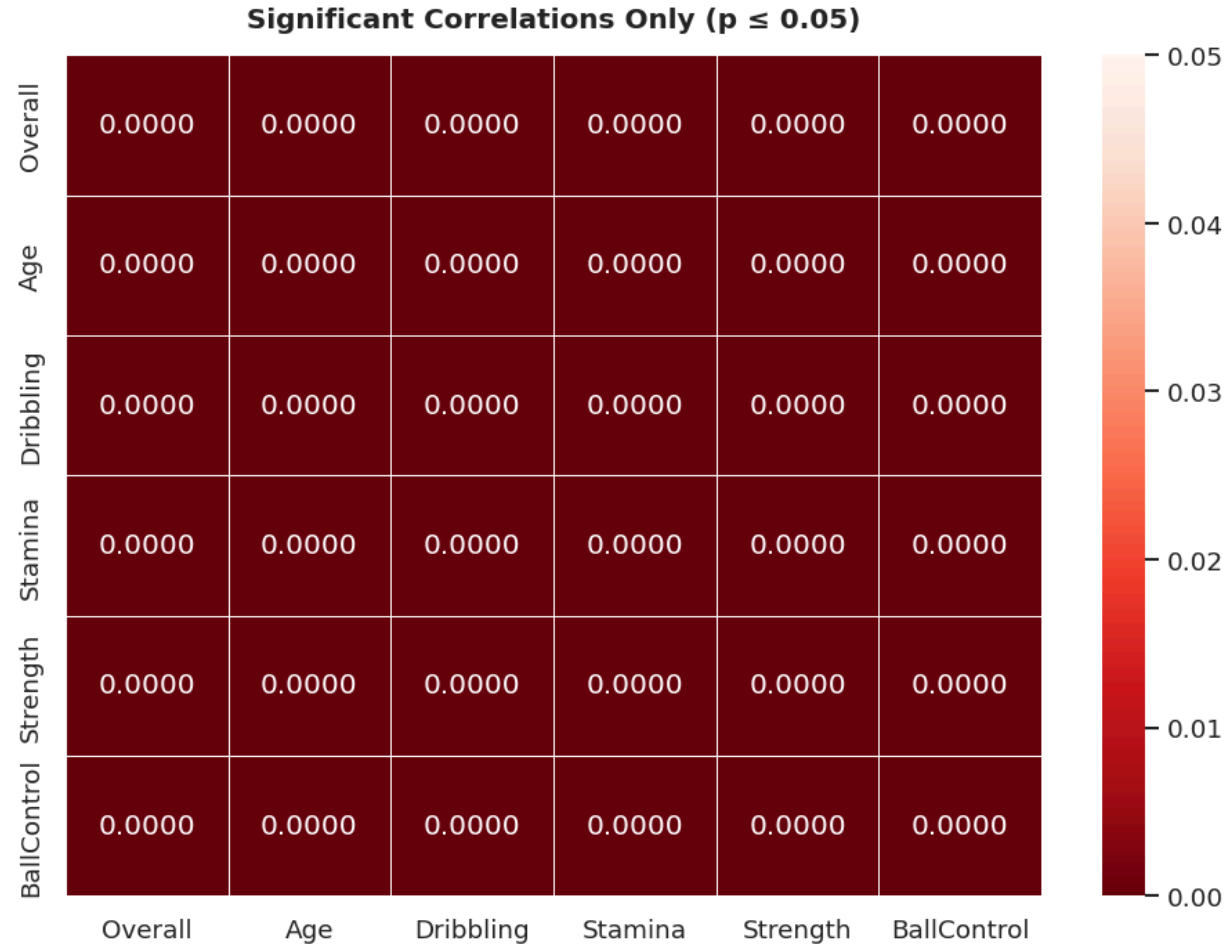
print(p_matrix.round(4))
print()

# Mask non-significant pairs (p > 0.05)
mask = p_matrix > 0.05

fig, ax = plt.subplots(figsize=(8, 6), dpi=120)
sns.heatmap(p_matrix, mask=mask, annot=True, fmt='.4f',
            cmap='Reds_r', vmin=0, vmax=0.05, linewidths=0.5, ax=ax)

ax.set_title('Significant Correlations Only (p ≤ 0.05)', fontsize=12, fontweight='bold', pad=12)
plt.tight_layout()
plt.show()
```

	Overall	Age	Dribbling	Stamina	Strength	BallControl
Overall	0.0	0.0	0.0	0.0	0.0	0.0
Age	0.0	0.0	0.0	0.0	0.0	0.0
Dribbling	0.0	0.0	0.0	0.0	0.0	0.0
Stamina	0.0	0.0	0.0	0.0	0.0	0.0
Strength	0.0	0.0	0.0	0.0	0.0	0.0
BallControl	0.0	0.0	0.0	0.0	0.0	0.0



☐ Task 6 — Categorical Relationships (Chi-Square)

6a. Is Preferred Foot linked to Position Group?

```
# H0: Preferred foot and position group are independent (not related)
# H1: There is a relationship between preferred foot and position group

# Crosstab (counts)
```

```
table = pd.crosstab(df['Preferred Foot'], df['pos_group'])
print("Frequency Table:")
print(table)
print()

# Row percentages
row_pct = table.div(table.sum(axis=1), axis=0) * 100
print("Row Percentages (%):")
print(row_pct.round(2))
print()

# Chi-Square test
chi2, p_val, dof, expected = stats.chi2_contingency(table)

print(f"Chi-Square Statistic: {chi2:.4f}")
print(f"Degrees of Freedom:  {dof}")
print(f"p-value:              {p_val:.4e}")
print()

# Verdict & Interpretation
if p_val < 0.05:
    print("Verdict: Reject H0 — There is a statistically significant relationship between Preferred Foot and Position Group.")
else:
    print("Verdict: Fail to Reject H0 — Preferred Foot and Position Group are independent.")
```

```
Frequency Table:
pos_group       Defender  Forward    GK  Midfielder
Preferred Foot
Left                1884      596   206        1523
Right               3982     2822  1819        5315

Row Percentages (%):
pos_group       Defender  Forward     GK  Midfielder
Preferred Foot
Left               44.76    14.16   4.89       36.18
Right              28.57    20.25  13.05       38.13

Chi-Square Statistic: 521.7735
Degrees of Freedom:  3
p-value:              9.1168e-113

Verdict: Reject H0 — There is a statistically significant relationship between Preferred Foot and Position Group.
```

6b. Is Skill Moves Level Linked to Position Group?

```
# 6b – Skill Tier vs Position Group

# H0: Skill tier and position group are independent (not related).
# H1: There is a relationship between skill tier and position group.

# Create skill_tier column
df['skill_tier'] = np.where(df['Skill Moves'] >= 4, 'High', 'Low')

# Crosstab (counts)
table_skill = pd.crosstab(df['skill_tier'], df['pos_group'])
print("Frequency Table:")
```

```
print(table_skill)
print()

# Row percentages
row_pct_skill = table_skill.div(table_skill.sum(axis=1), axis=0) * 100
print("Row Percentages (%):")
print(row_pct_skill.round(2))
print()

# Chi-Square test
chi2, p_val, dof, expected = stats.chi2_contingency(table_skill)

print(f"Chi-Square Statistic: {chi2:.4f}")
print(f"Degrees of Freedom:  {dof}")
print(f"p-value:              {p_val:.4e}")
print()

# Verdict & Interpretation
if p_val < 0.05:
    print("Verdict: Reject H0 — There is a statistically significant relationship between Skill Tier and Position Group.")
else:
    print("Verdict: Fail to Reject H0 — Skill Tier and Position Group are independent.")
```

```
Frequency Table:
pos_group   Defender  Forward    GK  Midfielder
skill_tier
High              48      312     0         608
Low             5818     3106  2025        6230

Row Percentages (%):
pos_group   Defender  Forward     GK  Midfielder
skill_tier
High            4.96    32.23   0.00       62.81
Low            33.87    18.08  11.79       36.27

Chi-Square Statistic: 619.7948
Degrees of Freedom:  3
p-value:              5.1526e-134

Verdict: Reject H0 — There is a statistically significant relationship between Skill Tier and Position Group.
```

6c. Visualise the Crosstabs

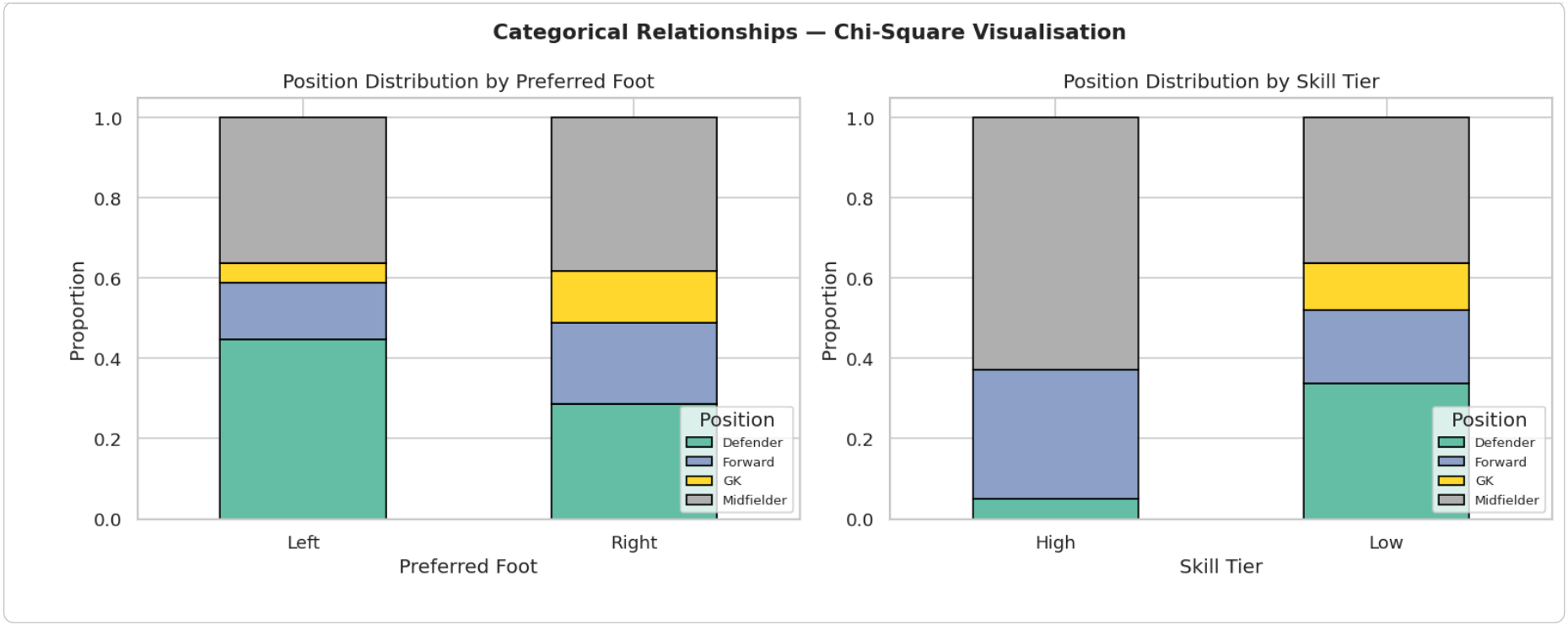
```
# 6c — Stacked Bar Charts
fig, axes = plt.subplots(1, 2, figsize=(13, 5), dpi=120)

# LEFT: Preferred Foot vs pos_group
foot_pct = pd.crosstab(df['Preferred Foot'], df['pos_group'])
foot_pct = foot_pct.div(foot_pct.sum(axis=1), axis=0)
foot_pct.plot(
    kind='bar',
    stacked=True,
    ax=axes[0],
    colormap='Set2',
    edgecolor='black',
)
```

```
axes[0].set_title('Position Distribution by Preferred Foot')
axes[0].set_xlabel('Preferred Foot')
axes[0].set_ylabel('Proportion')
axes[0].legend(title='Position', loc='lower right', fontsize=8)
axes[0].tick_params(axis='x', rotation=0)

# RIGHT: Skill Tier vs pos_group
skill_pct = pd.crosstab(df['skill_tier'], df['pos_group'])
skill_pct = skill_pct.div(skill_pct.sum(axis=1), axis=0)
skill_pct.plot(
    kind='bar',
    stacked=True,
    ax=axes[1],
    colormap='Set2',
    edgecolor='black',
)
axes[1].set_title('Position Distribution by Skill Tier')
axes[1].set_xlabel('Skill Tier')
axes[1].set_ylabel('Proportion')
axes[1].legend(title='Position', loc='lower right', fontsize=8)
axes[1].tick_params(axis='x', rotation=0)

plt.suptitle(
    'Categorical Relationships — Chi-Square Visualisation',
    fontsize=13,
    fontweight='bold',
)
plt.tight_layout()
plt.show()
```



📋 Task 7 — Summary & Final Visualisation

7a. Results Summary Table

```
# 7a – Summary Table
results = [
    {
        'Test': 't-test',
        'Variables': 'Preferred Foot → Overall',
        'Statistic': 5.8904,
        'p_value': 3.9201e-09,
        'Significant': True,
        'Verdict': 'Reject H0',
    },
    {
        'Test': 't-test',
        'Variables': 'Forwards vs Defenders → Overall',
        'Statistic': -0.0215,
        'p_value': 9.8285e-01,
        'Significant': False,
        'Verdict': 'Fail to Reject H0',
    },
    {
        'Test': 'ANOVA',
        'Variables': 'pos_group → Overall',
```

```
    'Statistic': 44.2100,
    'p_value': 1.8599e-28,
    'Significant': True,
    'Verdict': 'Reject H0',
},
{
    'Test': 'Spearman',
    'Variables': 'BallControl ↔ Overall',
    'Statistic': 0.6228,
    'p_value': 0.0000e00,
    'Significant': True,
    'Verdict': 'Reject H0',
},
{
    'Test': 'Spearman',
    'Variables': 'Age ↔ Potential',
    'Statistic': -0.2623,
    'p_value': 3.6277e-283,
    'Significant': True,
    'Verdict': 'Reject H0',
},
{
```